COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	9/13/2026	**DATE SUBMITTED:**	9/14/2026
NAME:	Denzen D. Igloso	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. __8_ SORTING ALGORITHMS: SHELL, MERGE, AND QUICK SORT

1. PROCEDURE OUTPUT

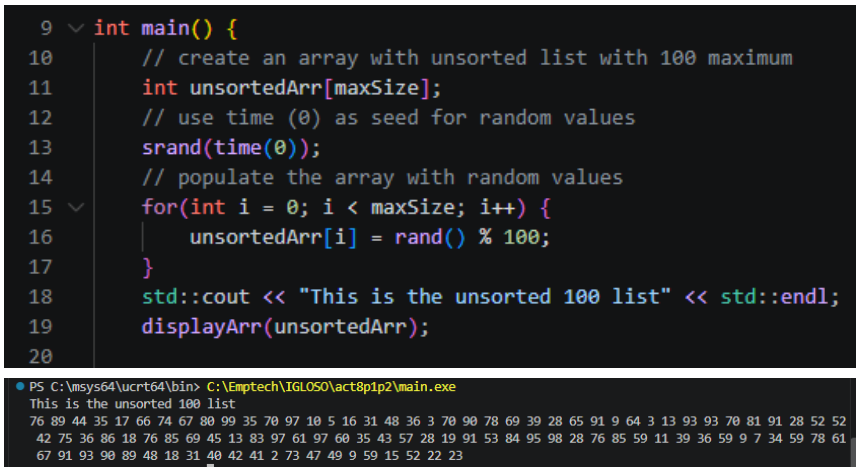
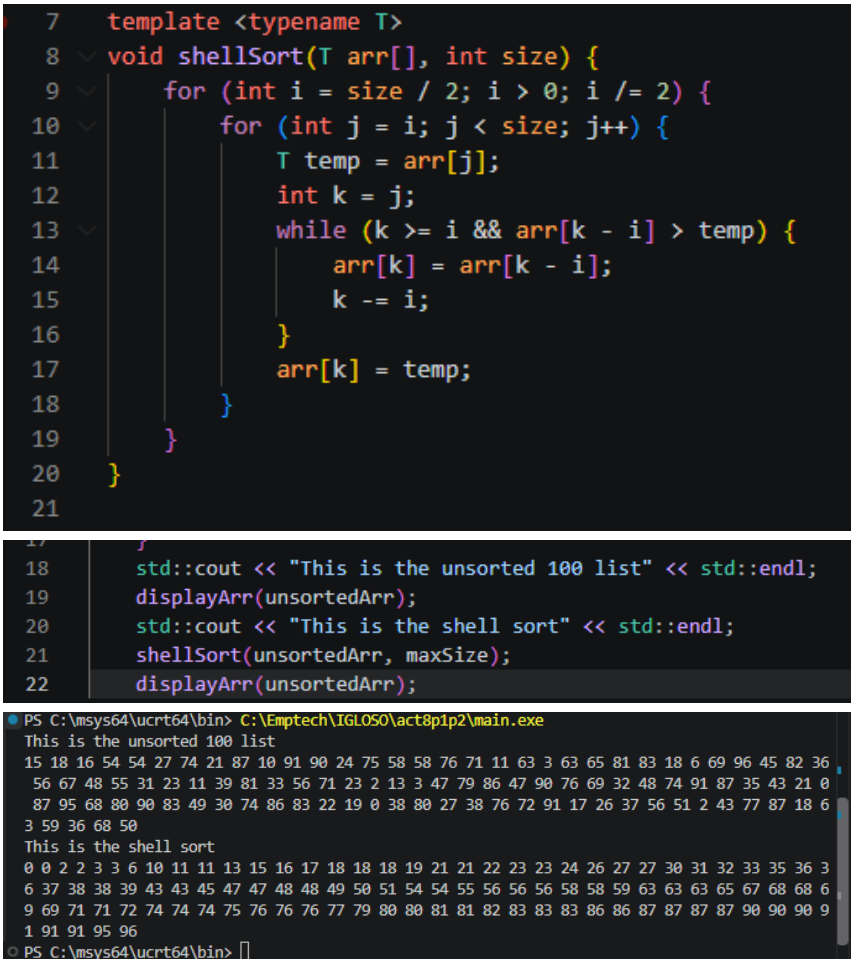
Code + Console Screenshot	
Observations	This part generates 100 random numbers using rand() and puts them in an array.

Table 8-1. Array of Values for Sort Algorithm Testing

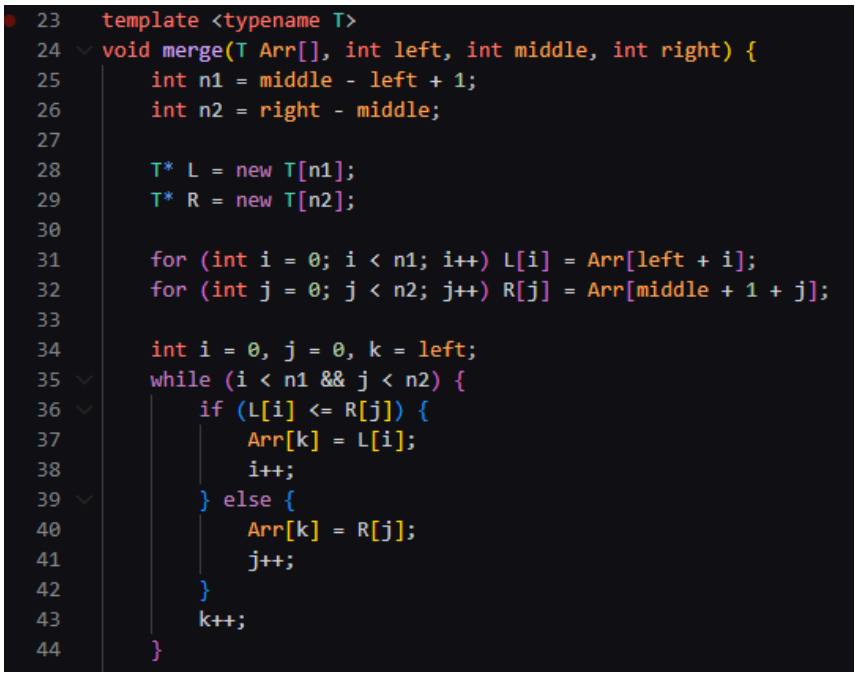
Code + Console Screenshot	
Observations	The algorithm segregates the array into different sub-lists using an interval gap starting from half the length of the array, which is reduced by half after each pass. The elements positioned in intervals equal to gap distance are then sorted using insertion sort technique, which helps in moving disordered elements much faster in the array.

```
template <typename T>
void shellSort(T arr[], int size) {
    for (int i = size / 2; i > 0; i /= 2) {
        for (int j = i; j < size; j++) {
            T temp = arr[j];
            int k = j;
            while (k >= i && arr[k - i] > temp) {
                arr[k] = arr[k - i];
                k -= i;
            }
            arr[k] = temp;
        }
    }
}
```

```
    std::cout << "This is the unsorted 100 list" << std::endl;
    displayArr(unsortedArr);
    std::cout << "This is the shell sort" << std::endl;
    shellSort(unsortedArr, maxSize);
    displayArr(unsortedArr);
```

```
PS C:\msys64\ucrt64\bin> C:\Emptech\IGLOSO\act8p1p2\main.exe
This is the unsorted 100 list
15 18 16 54 54 27 74 21 87 10 91 90 24 75 58 58 76 71 11 63 3 63 65 81 83 18 6 69 96 45 82 36
 56 67 48 55 31 23 11 39 81 33 56 71 23 2 13 3 47 79 86 47 90 76 69 32 48 74 91 87 35 43 21 0
 87 95 68 80 90 83 49 30 74 86 83 22 19 0 38 80 27 38 76 72 91 17 26 37 56 51 2 43 77 87 18 6
3 59 36 68 50
This is the shell sort
0 0 2 2 3 3 6 10 11 11 13 15 16 17 18 18 18 19 21 21 22 23 23 24 26 27 27 30 31 32 33 35 36 3
6 37 38 38 39 43 43 45 47 47 48 48 49 50 51 54 54 55 56 56 56 58 58 59 63 63 63 65 67 68 68 6
9 69 71 71 72 74 74 74 75 76 76 76 77 79 80 80 81 81 82 83 83 83 86 86 87 87 87 87 90 90 90 9
1 91 91 95 96
PS C:\msys64\ucrt64\bin>
```

Table 8-2. Shell Sort Algorithm

Code + Console Screenshot	

```
23  template <typename T>
24  void merge(T Arr[], int left, int middle, int right) {
25      int n1 = middle - left + 1;
26      int n2 = right - middle;
27
28      T* L = new T[n1];
29      T* R = new T[n2];
30
31      for (int i = 0; i < n1; i++) L[i] = Arr[left + i];
32      for (int j = 0; j < n2; j++) R[j] = Arr[middle + 1 + j];
33
34      int i = 0, j = 0, k = left;
35      while (i < n1 && j < n2) {
36          if (L[i] <= R[j]) {
37              Arr[k] = L[i];
38              i++;
39          } else {
40              Arr[k] = R[j];
41              j++;
42          }
43          k++;
44      }
```

```
46      while (i < n1) {
47          Arr[k] = L[i];
48          i++;
49          k++;
50      }
51
52      while (j < n2) {
53          Arr[k] = R[j];
54          j++;
55          k++;
56      }
57
58      delete[] L;
59      delete[] R;
60  }
61
62  template <typename T>
63  void merge_sort(T Arr[], int left, int right) {
64      if (left < right) {
65          int middle = (left + right) / 2;
66          merge_sort(Arr, left, middle);
67          merge_sort(Arr, middle + 1, right);
68          merge(Arr, left, middle, right);
69      }
70  }
```

```
18      std::cout << "This is the unsorted 100 list" << std::endl;
19      displayArr(unsortedArr);
20      std::cout << "This is the merge sort" << std::endl;
21      merge_sort(unsortedArr, 0, maxSize - 1);
22      displayArr(unsortedArr);
23  }
```

```
PS C:\msys64\ucrt64\bin> C:\Emptech\IGLOSO\act8p1p2\main.exe
This is the unsorted 100 list
5 33 62 71 76 2 56 79 19 70 13 76 59 90 52 59 55 90 88 38 43 53 57 44 56 36 18 19 64 31 97 83
 56 15 17 75 92 1 50 19 48 89 6 94 84 38 86 57 13 21 36 75 73 85 56 78 47 42 9 22 21 29 26 69
 62 62 92 87 27 59 49 50 20 33 21 53 51 12 68 36 36 7 52 64 95 20 45 92 6 1 18 11 31 8 78 44
93 49 71 6
This is the merge sort
1 1 2 5 6 6 6 7 8 9 11 12 13 13 15 17 18 18 19 19 19 20 20 21 21 21 22 26 27 29 31 31 33 33 3
6 36 36 36 38 38 42 43 44 44 45 47 48 49 49 50 50 51 52 52 53 53 55 56 56 56 56 57 57 59 59 5
9 62 62 62 64 64 68 69 70 71 71 73 75 75 76 76 78 78 79 83 84 85 86 87 88 89 90 90 92 92 92 9
3 94 95 97
PS C:\msys64\ucrt64\bin>
```

Observations	The sorting algorithm divides the list into two halves recursively, splitting the lists until the size of each sub-list becomes one element. The algorithm then merges these sub-lists back together after comparing them to obtain the sorted list.

Table 8-3. Merge Sort Algorithm

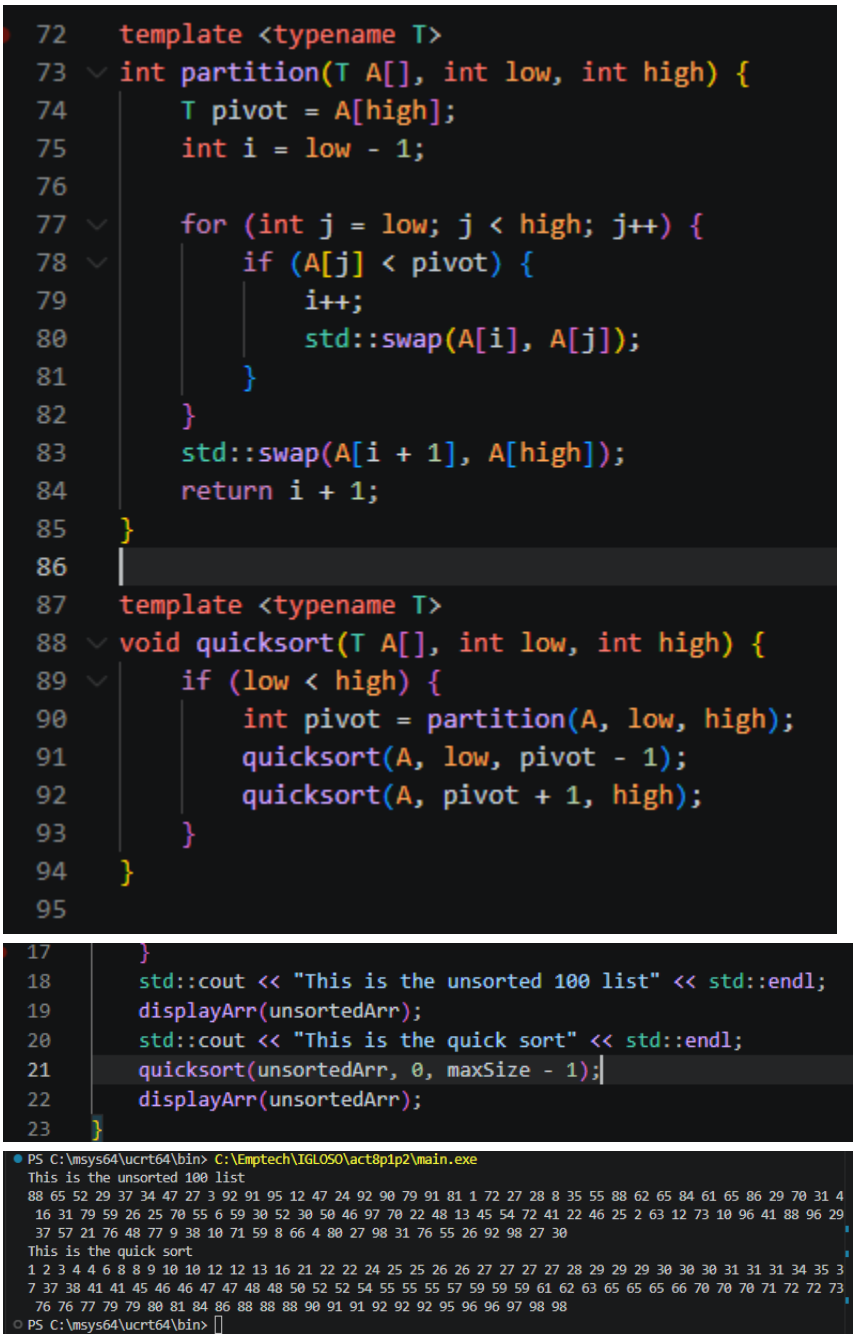
Code + Console Screenshot	
Observations	In this algorithm, the choice of pivot is made based on the last item. The array will be divided into two sub-arrays, such that values that are less than the pivot are kept to the left, whereas those values that are larger than the pivot are put to the right.

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

PROBLEM 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

- Yes, we can. Quick sort makes use of a partition technique to sort a selected pivot element into its actual location, thus resulting in the creation of two sub-lists. The elements on the left sub-list will be those that are less than the pivot value while those on the right will be greater than the pivot value. These two sub-lists need not necessarily be sorted using Quick Sort; they may be sorted using any other sorting method.
- Given Array: {10, 42, 15, 12, 37, 7, 9}
- Step 1 (Partition using Pivot = 9):
    - Left side which the elements smaller than 9: {7}
    - Pivot element in correct position: 9
    - Right side which the elements larger than 9: {10, 42, 15, 12, 37}
- Step 2 (Sort sub-lists using Insertion Sort):
    - Left {7} contains 1 element, so it is already sorted
    - Right {10, 42, 15, 12, 37} sorted using Insertion Sort becomes {10, 12, 15, 37, 42}
- Final Result: Combined sorted array is {7, 9, 10, 12, 15, 37, 42}.

PROBLEM 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}.

Question 1: What sorting algorithm will give you the fastest time performance?

For the array {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}, Quick Sort will deliver the fastest time performance. While both Quick Sort and Merge Sort share an optimal average time complexity, Quick Sort performs fewer unnecessary copy operations because it sorts the array in-place rather than allocating secondary storage. This in-place partitioning gives Quick Sort much lower memory overhead and significantly better CPU cache performance, allowing it to complete the sorting process faster in practice.

Question 2: Why can merge sort and quick sort have O(N • log N) for their time complexity?

Merge Sort and Quick Sort achieve an O(N log N) time complexity because of how their divide-and-conquer strategy divides the work. The log N factor represents the depth of the recursion tree, as repeatedly dividing an array of size N in half requires log2 N levels of splitting until reaching single-element sub-lists. The N factor comes from the total work performed across each level, where every element in the array is examined during the merging or partitioning phases. Multiplying the O(N) operations done at each level by the log N total levels produces an overall time complexity of O(N log N).

CONCLUSION

- In conclusion, the performance of these sorts of algorithms proves the efficiency of divide-and-conquer technique with respect to big data sets as opposed to loop-based sorting algorithms. It may be easily seen that while algorithms like Insertion and Selection are straightforward and effective for sorting small arrays, more sophisticated algorithms like Quick Sort and Merge Sort save a lot of time with the increasing amount of data elements. Thus, one needs to choose an algorithm according to specific requirements of a particular task.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.